Gateway API

What is Gateway API?

Gateway API is the modern Kubernetes API for managing how external traffic enters a cluster and reaches applications.

It is designed as the **successor to Ingress**, fixing Ingress's limitations while supporting real-world production needs like:

- Advanced routing
- gRPC traffic
- Multi-team clusters
- Clean, portable configuration

In short:

> Ingress was good for the early days of Kubernetes. Gateway API is built for today's large-scale, multi-team systems.

Why Gateway API Was Created

Ingress had several fundamental problems:

- One resource tried to do everything
- Heavy reliance on annotations
- No clear separation of responsibilities
- Poor support for advanced routing
- Controller-specific behavior
- Hard to manage safely in large teams

Gateway API was created to **solve these problems by design**, not with workarounds.

The Core Design Shift

Ingress mindset:

> "Put all networking logic into one YAML file."

Gateway API mindset:

> "Different people manage different responsibilities."

This **role-oriented design** is the biggest improvement.

Role-Oriented Design (Ingress vs Gateway API)

Ingress (No clear roles)

- Infra team edits Ingress
- Platform team edits Ingress
- App team edits Ingress

  ➡ Everyone touches the same resource

  ➡ High risk of mistakes and outages

Gateway API (Clear ownership)

Role	Responsibility
Infrastructure Provider	Load balancers and gateway infrastructure
Cluster Operator	Security, access, policies
Application Developer	Routing rules to services

Each team works **independently and safely**.

Gateway API Components (Simple Mental Model)

Real-World Analogy: Office Building

Kubernetes Object	Analogy	What it does
GatewayClass	Building blueprint	Defines which controller manages gateways
Gateway	Building entrance	Where traffic enters (ports, protocols)
HTTPRoute	Reception desk rules	Routes HTTP traffic based on host/path/headers
GRPCRoute	gRPC reception	Routes gRPC traffic by service/method

---

Resource Model (Clean and Layered)

```
GatewayClass → Gateway → Route → Service
```

Each layer has **one clear purpose**, unlike Ingress where everything is mixed together.

Why Gateway API Is Better Than Ingress (Deep Comparison)

1. Structure: Flat vs Layered

Ingress

- Single object
- Hard to read
- Hard to review
- Easy to break

Gateway API

- Clean separation
- Easier debugging
- Safer changes
- Better scalability

---

2. Annotations vs Real Fields

Ingress

- Annotations control behavior
- Not standardized
- Controller-specific
- Hard to migrate

Gateway API

- Uses proper API fields
- Standard across implementations
- Portable and readable
- Validated by Kubernetes

---

3. Advanced Traffic Routing

Ingress limitations

- Weak header-based routing
- No native traffic weighting

- Poor gRPC support
- Requires hacks and custom annotations

Gateway API

- Header and path matching
- Traffic splitting (canary, blue/green)
- Native gRPC routing
- Multiple backends per route

All **built-in**, no hacks.

---

4. gRPC Support (Major Upgrade)

Ingress

- Treats gRPC as HTTP
- Breaks easily
- Needs controller-specific config

Gateway API

- Native `GRPCRoute`
- Guaranteed HTTP/2 support
- Can route by:
  - gRPC service
  - gRPC method

This is **not realistically possible with Ingress**.

---

5. Security & Multi-Tenancy

Ingress

- Any team can modify hosts and paths
- Easy to break or expose other apps

Gateway API

- Explicit attachment rules
- Gateway controls which routes can attach
- Bidirectional trust model

This makes Gateway API **safe for shared clusters**.

---

6. Extensibility & Future-Proofing

Ingress

- Feature-frozen
- New behavior = annotations

Gateway API

- Designed to be extended
- Supports custom policies
- Clean integration points

Gateway API can grow **without breaking users**.

---

7. Portability Across Controllers

Ingress

- YAML changes per controller
- Vendor lock-in

Gateway API

- Same routes across controllers
- Only GatewayClass changes
- Truly vendor-neutral

Request flow

Here is a simple example of HTTP traffic being routed to a Service by using a Gateway and an HTTPRoute:

A diagram that provides an example of HTTP traffic being routed to a Service by using a Gateway and an HTTPRoute

In this example, the request flow for a Gateway implemented as a reverse proxy is:

1. The client starts to prepare an HTTP request for the URL `http://www.example.com`
2. The client's DNS resolver queries for the destination name and learns a mapping to one or more IP addresses associated with the Gateway.
3. The client sends a request to the Gateway IP address; the reverse proxy receives the HTTP request and uses the Host: header to match a configuration that was derived from the Gateway and attached HTTPRoute.

4. Optionally, the reverse proxy can perform request header and/or path matching based on match rules of the HTTPRoute.
5. Optionally, the reverse proxy can modify the request; for example, to add or remove headers, based on filter rules of the HTTPRoute.
6. Lastly, the reverse proxy forwards the request to one or more backends.

Real-World Analogy: Traffic System

Ingress

- Single traffic cop
- Limited signals
- Works until traffic grows

Gateway API

- Smart city traffic system
- Roads, signals, and rules are separate
- Scales cleanly with growth

Is Gateway API Replacing Ingress?

Yes.

- Ingress is still supported
- But feature development has moved to Gateway API
- New projects should use Gateway API

When to Use What

Use Ingress only if:

- Small cluster
- Single team
- Very simple routing

Use Gateway API if:

- Multiple teams
- Microservices
- gRPC workloads
- Advanced routing needs
- Production-grade Kubernetes

Final Summary (One Paragraph)

Gateway API is the modern, role-based replacement for Ingress.

It separates infrastructure, access, and routing concerns into clean layers, removes annotation chaos, supports advanced HTTP and gRPC routing, and enables safe multi-team usage. While Ingress solved early Kubernetes needs, Gateway API is built for today's large-scale, production environments

Resource model

Gateway API has four stable API kinds:

- **GatewayClass:** Defines a set of gateways with common configuration and managed by a controller that implements the class.
- **Gateway:** Defines an instance of traffic handling infrastructure, such as cloud load balancer.
- **HTTPRoute:** Defines HTTP-specific rules for mapping traffic from a Gateway listener to a representation of backend network endpoints. These endpoints are often represented as a Service.
- **GRPCRoute:** Defines gRPC-specific rules for mapping traffic from a Gateway listener to a representation of backend network endpoints. These endpoints are often represented as a Service.

Gateway API is organized into different API kinds that have interdependent relationships to support the role-oriented nature of organizations. A Gateway object is associated with exactly one GatewayClass; the GatewayClass describes the gateway controller responsible for managing Gateways of this class. One or more route kinds such as HTTPRoute, are then associated to Gateways. A Gateway can filter the routes that may be attached to its `listeners`, forming a bidirectional trust model with routes.

The following figure illustrates the relationships of the three stable Gateway API kinds:

![A figure illustrating the relationships of the three stable Gateway API kinds](https://kubernetes.io/docs/images/gateway-kind-relationships.svg)

A figure illustrating the relationships of the three stable Gateway API kinds

GatewayClass

Gateways can be implemented by different controllers, often with different configurations. A Gateway must reference a GatewayClass that contains the name of the controller that implements the class.

A minimal GatewayClass example:

```
**apiVersion**: gateway.networking.k8s.io/v1
**kind**: GatewayClass
```

```
**metadata**:
  **name**: example-class
**spec**:
  **controllerName**: example.com/gateway-controller
```

In this example, a controller that has implemented Gateway API is configured to manage GatewayClasses with the controller name `example.com/gateway-controller`. Gateways of this class will be managed by the implementation's controller.

Gateway

A Gateway describes an instance of traffic handling infrastructure. It defines a network endpoint that can be used for processing traffic, i.e. filtering, balancing, splitting, etc. for backends such as a Service. For example, a Gateway may represent a cloud load balancer or an in-cluster proxy server that is configured to accept HTTP traffic.

A minimal Gateway resource example:

```
**apiVersion**: gateway.networking.k8s.io/v1
**kind**: Gateway
**metadata**:
  **name**: example-gateway
**spec**:
  **gatewayClassName**: example-class
  **listeners**:
  - **name**: http
    **protocol**: HTTP
    **port**: 80
```

In this example, an instance of traffic handling infrastructure is programmed to listen for HTTP traffic on port 80. Since the `addresses` field is unspecified, an address or hostname is assigned to the Gateway by the implementation's controller. This address is used as a network endpoint for processing traffic of backend network endpoints defined in routes.

HTTPRoute

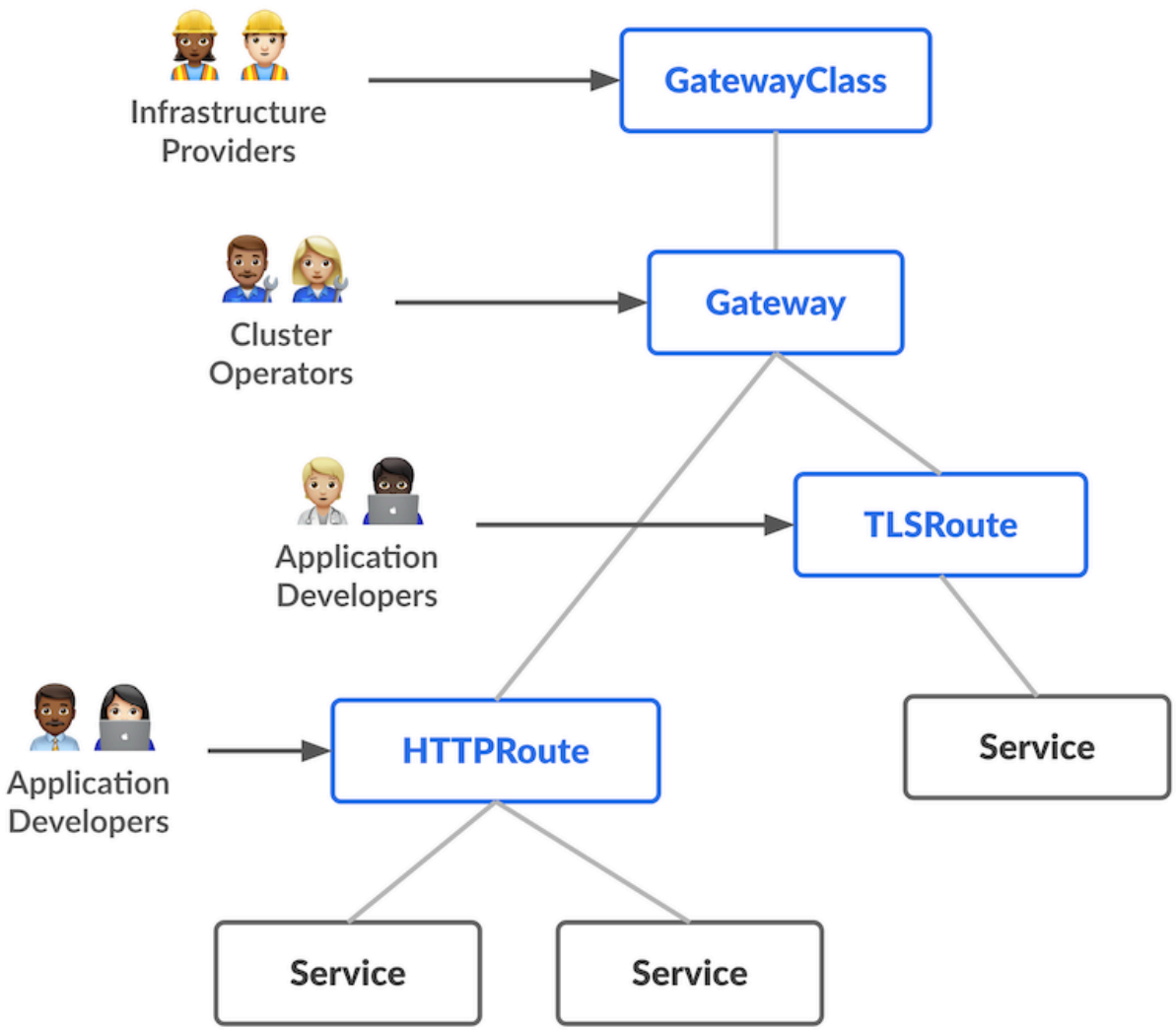
The HTTPRoute kind specifies routing behavior of HTTP requests from a Gateway listener to backend network endpoints. For a Service backend, an implementation may represent the backend network endpoint as a Service IP or the backing EndpointSlices of the Service. An HTTPRoute represents configuration that is applied to the underlying Gateway implementation. For example, defining a new HTTPRoute may result in configuring additional traffic routes in a cloud load balancer or in-cluster proxy server.

A minimal HTTPRoute example:

```
**apiVersion**: gateway.networking.k8s.io/v1
**kind**: HTTPRoute
**metadata**:
  **name**: example-httproute
**spec**:
```

```
**parentRefs**:
- **name**: example-gateway
**hostnames**:
- "www.example.com"
**rules**:
- **matches**:
  - **path**:
      **type**: PathPrefix
      **value**: /login
  **backendRefs**:
  - **name**: example-svc
    **port**: 8080
```

In this example, HTTP traffic from Gateway `example-gateway` with the Host: header set to `www.example.com` and the request path specified as `/login` will be routed to Service `example-svc` on port `8080`.


Infrastructure Providers
GatewayClass
Cluster Operators
Gateway
Application Developers
TLSRoute
Application Developers
HTTPRoute
Service
Service
Service

Project Setup

We will deploy an application called `httpbin` and will expose using gateway api.

As we now know when we deploy the controller by default it creates a gatway class.

We will use that gateway class.

About the Application

The httpbin app lets you test your API gateway by sending requests to it and receiving responses.

Review the following diagram to understand the setup.

```
client →www.example.com →gateway proxy → httpbin backend
```

- The client calls the `www.example.com` hostname that you set up in the Gateway configuration.
- The gateway proxy receives the request. Based on the routing rules that you set up in the Gateway configuration, the gateway proxy forwards the traffic to the backend destination, which is the httpbin service. The gateway proxy is available from an external LoadBalancer service that is backed by an IP address that your cloud provider typically assigns. For testing in a local cluster where you do not have an external service, you can enable port-forwarding so that the gateway proxy listens on the localhost instead.
- The httpbin service receives and responds to the request. Note that the httpbin service does not have to be publicly exposed because the gateway proxy handles the external traffic. Instead, it can have an internal service type, such as ClusterIP.

1. Create the httpbin app.

   ```
   kubectl apply -f
   https://raw.githubusercontent.com/kgateway-dev/kgateway/refs/heads/v2.1
   .x/examples/httpbin.yaml
   ```

   Example output:

   ```
   namespace/httpbin created
   serviceaccount/httpbin created
   service/httpbin created
   deployment.apps/httpbin created
   ```

2. Verify that the httpbin app is running.

   ```
   kubectl -n httpbin get pods
   ```

   Example output:

```
NAME                        READY   STATUS    RESTARTS   AGE
httpbin-d57c95548-nz98t     2/2     Running   0          18s
```

Set up an API gateway

Create an API gateway with an HTTP listener by using the Kubernetes Gateway API.

1. Create a Gateway resource and configure an HTTP listener. The following Gateway can serve HTTPRoute resources from all namespaces.

   ```
   kubectl apply -f- <<EOF
   kind: Gateway
   apiVersion: gateway.networking.k8s.io/v1
   metadata:
     name: http
     namespace: kgateway-system
   spec:
     gatewayClassName: kgateway
     listeners:
     - protocol: HTTP
       port: 80
       name: http
       allowedRoutes:
         namespaces:
           from: All
   EOF
   ```

2. Verify that the Gateway is created successfully. You can also review the external address that is assigned to the Gateway. Note that depending on your environment it might take a few minutes for the load balancer service to be assigned an external address.

   You can check the AWS console and make sure its provisioned successfully.

   ```
   kubectl get gateway http -n kgateway-system
   ```

   Example output:

   ```
   NAME   CLASS      ADDRESS                                    PROGRAMMED
   AGE
   http   kgateway   1234567890.us-east-2.elb.amazonaws.com   True
   93s
   ```

3. Verify that the gateway proxy pod is running.

   ```
   kubectl get po -n kgateway-system -l
   gateway.networking.k8s.io/gateway-name=http
   ```

   Example output:

```
NAME                    READY   STATUS    RESTARTS   AGE
http-7dd94b74-k26j6     3/3     Running   0          18s
```

Expose the app on the gateway

Now that you have an app and a gateway proxy, you can create a route to access the app.

1. Create an HTTPRoute resource to expose the httpbin app on the Gateway. The following example exposes the app on the `wwww.example.com` domain.

```
kubectl apply -f- <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: httpbin
  namespace: httpbin
spec:
  parentRefs:
    - name: http
      namespace: kgateway-system
  hostnames:
    - "httpbin.dsvault.in"
  rules:
    - backendRefs:
        - name: httpbin
          port: 8000
EOF
```

Setting	Description
`spec.parentRefs`	The name and namespace of the Gateway resource that serves the route. In this example, you use the `http` Gateway that you created earlier.
`spec.hostnames`	A list of hostnames that the route is exposed on. In the example, the route is exposed on `www.example.com`.
`spec.rules.backendRefs`	The Kubernetes service that serves the incoming request. In this example, requests to `www.example.com` are forwarded to the httpbin app on port 8000. Note that you must create the HTTPRoute in the same namespace as the service that serves that route. To create the HTTPRoute resource

Setting	Description
	in a different namespace, you must create a ReferenceGrant resource to allow the HTTPRoute to forward requests to a service in a different namespace.

2. Verify that the HTTPRoute is applied successfully.

```
kubectl get -n httpbin httproute/httpbin -o yaml
```

Example output: Note the status of the HTTPRoute resource. Check for `Accepted` and `ResolvedRefs` messages. The `parentRef` refers to the Gateway that that HTTPRoute is exposed on.

```
status:
  parents:
  - conditions:
    - lastTransitionTime: "2025-02-13T18:41:06Z"
      message: ""
      observedGeneration: 1
      reason: Accepted
      status: "True"
      type: Accepted
    - lastTransitionTime: "2025-02-13T18:41:06Z"
      message: ""
      observedGeneration: 1
      reason: ResolvedRefs
      status: "True"
      type: ResolvedRefs
    controllerName: kgateway.dev/kgateway
    parentRef:
      group: gateway.networking.k8s.io
      kind: Gateway
      name: http
      namespace: kgateway-system
```

Send a request

Now that your httpbin app is running and exposed on the gateway proxy, you can send a request to the app. The steps vary depending on your load balancer setup.

1. Get the external address of the gateway proxy and save it in an environment variable or manually copy from AWS console.

```
export INGRESS_GW_ADDRESS=$(kubectl get svc -n kgateway-system http
-o=jsonpath="{.status.loadBalancer.ingress[0]['hostname','ip']}")
echo $INGRESS_GW_ADDRESS
```

2. Send a request to the httpbin app and verify that you get back a 200 HTTP response code. Note that it might take a few seconds for the load balancer service to become fully ready and accept traffic.

   ```
   curl -i http://$INGRESS_GW_ADDRESS:8080/headers -H "host:
   httpbin.dsvault.in:80"
   ```

   Example output:

   ```
   **HTTP/1.1 200 OK
   server: envoy
   date: Wed, 17 Jan 2024 17:32:21 GMT
   content-type: application/json
   content-length: 211
   access-control-allow-origin: *
   access-control-allow-credentials: true
   x-envoy-upstream-service-time: 2**
   ```

3. To get access in the browser add the DNS record in your DNS zone.

   ```
   httpbin.dsvault.in -> your load balancer dns name
   ```

4. Now access with the hostname `httpbin.dsvault.in`.

Cleanup Project

You can remove the resources that you created.

1. Delete the httpbin app.

   ```
   kubectl delete -f
   https://raw.githubusercontent.com/kgateway-dev/kgateway/refs/heads/v2.1
   .x/examples/httpbin.yaml
   ```

2. Delete the HTTPRoute.

   ```
   kubectl delete httproute httpbin -n httpbin
   ```

3. Delete the Gateway.

   ```
   kubectl delete gateway http -n kgateway-system
   ```

Gateway API Implementations

Just like:

- Kubernetes defines **Ingress**
- NGINX / ALB / Traefik **implement Ingress**

Gateway API

- Defined by Kubernetes SIG Network
- Describes **WHAT objects exist**
  - GatewayClass
  - Gateway
  - HTTPRoute
  - GRPCRoute
- Does **not** handle traffic itself

➡ It's just **rules and APIs**

What does "Implementation" mean?

Gateway API itself does not route traffic.

An **implementation (controller)** is the software that:

- Watches Gateway API resources (Gateway, HTTPRoute, etc.)
- Creates/configures real networking components
- Actually routes traffic

👉 Examples: **kgateway, Istio, Envoy Gateway, NGINX Gateway Fabric**

Conformance Levels (How "correct" an implementation is)

Gateway API defines tests. Implementations are classified based on how well they pass them.

1. Conformant

- Passed **all required core tests**
- Passed **all extended features they claim**
- Tested against **recent Gateway API versions**
- Production-ready

✅ **Best and safest choice**

---

2. Partially Conformant

- Passed **some**, but not all, required tests
- Still working toward full conformance

⚠️ Usable, but may have gaps

---

3. Stale

- No recent conformance reports
- May not be actively maintained
- Will be removed from the list if not updated

❌ Avoid for new setups

Gateway Controller Implementation Status

Conformant (Recommended)

- Envoy Gateway
- Istio
- **kgateway**
- NGINX Gateway Fabric
- Traefik
- Cilium

Partially Conformant

- GKE Gateway
- Kong (Controller & Operator)
- Contour
- Gloo Gateway
- Azure Application Gateway for Containers

Stale (Not Recommended)

- Amazon EKS Gateway
- Apache APISIX
- HAProxy Ingress
- Consul
- Kuma
- ngrok

- Tyk
- WSO2 APK

  (and others)

To read more about it follow the below doc.

https://gateway-api.sigs.k8s.io/implementations/#kgateway

Here for this tutorial we will be using kagteway.

Kgateway is a control plane that implements the Kubernetes Gateway API for both microservices and AI workloads. The control plane translates your Kubernetes Gateway API resources into the configuration that the underlying data plane proxy can understand. The proxy layer is handled by kgateway's implementation of Envoy for microservices or agentgateway for AI workloads.

Installation

Install the kgateway control plane by using Helm.

1. Install the custom resources of the Kubernetes Gateway API version 1.4.0.Example output:

   ```
   **kubectl apply -f
   https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0
   /standard-install.yaml**
   ```

   Example Output:

   ```
   customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.ne
   tworking.k8s.io created
   customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networki
   ng.k8s.io created
   customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networ
   king.k8s.io created
   customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.n
   etworking.k8s.io created
   customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networ
   king.k8s.io created
   ```

2. Apply the kgateway CRDs for the upgrade version by using Helm.

   Deploy the kgateway CRDs by using Helm. This command creates the kgateway-system namespace and creates the kgateway CRDs in the cluster.

   ```
   helm upgrade -i --create-namespace \
     --namespace kgateway-system \
     --version v2.1.1 kgateway-crds
   oci://cr.kgateway.dev/kgateway-dev/charts/kgateway-crds
   ```

3. Install the kgateway Helm chart.

   Install kgateway control plane by using Helm. If you modified the `values.yaml` file with custom installation values, add the `-f kgateway/values.yaml` flag.

   ```
   helm upgrade -i -n kgateway-system kgateway
   oci://cr.kgateway.dev/kgateway-dev/charts/kgateway \
   --version v2.1.1
   ```

   Example Output:

   ```
   NAME: kgateway
   LAST DEPLOYED: Thu Feb 13 14:03:51 2025
   NAMESPACE: kgateway-system
   STATUS: deployed
   ```

```
REVISION: 1
TEST SUITE: None
```

4. Verify that the control plane is up and running.

   ```
   kubectl get pods -n kgateway-system
   ```

   Example Output:

   ```
   NAME                                  READY   STATUS    RESTARTS   AGE
   kgateway-78658959cd-cz6jt             1/1     Running   0          12s
   ```

5. Verify that the `kgateway` GatewayClass is created. You can optionally take a look at how the GatewayClass is configured by adding the `o yaml` option to your command.

   ```
   kubectl get gatewayclass kgateway
   ```